CS 519 – Project 1, Part 2:

Saroop Makhija (ssm229)

1 Implementation

Both implementations partition an $N \times N$ matrix multiplication across P child processes, each computing a disjoint set of output rows. Matrix B is stored transposed so the inner dot-product uses stride-1 access. All experiments ran on a CloudLab `m510` node **Shared memory** (`IPC-shmem.c`): Matrices A, B^T, and C are allocated via `shmget/shmat`. Each child writes to non-overlapping rows of C, so **no synchronization is needed**.

Pipes (`IPC-pipe.c`): Each child gets a **dedicated result pipe** (eliminating the baseline's shared pipe + semaphore bottleneck). Children compute into a local buffer and write results in one shot. The parent reads each pipe directly into the correct offset of C. Pipe buffers are enlarged to 1 MiB via `fcntl(F_SETPIPE_SZ)`.

2 Baseline Profiling and Bottleneck Identification

I profiled the unoptimized code with `perf stat` (N=2000, 4 processes):

Table 1: Baseline vs. optimized performance (N=2000, 4 processes)

Metric	Shmem Base	Shmem Opt	Pipe Base	Pipe Opt
Runtime (s)	1.484	**0.992**	1.767	**1.072**
IPC	1.19	**1.92**	1.05	**1.76**
Cache misses	549 M	**34 M**	682 M	**40 M**
Cache miss rate	99.96%	100%*	99.83%	100%*
L1d load misses	560 M	**40 M**	691 M	**53 M**
Context switches	27	27	74	—

*Cache miss *rate* stays high but the absolute count dropped 16×—the remaining misses are compulsory (cold) misses that cannot be eliminated.

Bottlenecks identified: (1) Near-100% cache miss rate from a naive triple loop with no tiling resulting in each matrix being 100 MB for $N = 5000$, far exceeding the 12 MB L3. (2) Low IPC (1.05–1.19) due to pipeline stalls waiting on ram. (3) In the pipe baseline, a single shared result pipe protected by a semaphore serialized all children's writes.

`perf report` **overhead (optimized):** `multiply_rows` dominates at 95.8% (shmem) / 98.2% (pipe, inlined by `-O3`). Remaining time is kernel page faults (∼1.5%) and `memset` for zeroing C (<1%).

3 Optimizations

1. Tiled loop with correct ordering. I tile the computation with TILE_K=64, TILE_J=64, sized so the B^T sub-block (64×64×4 B = 16 KiB) fits in the 32 KiB L1d. The critical change was the loop order:

Broken: `kk` → `i` → `jj` → `j` → `k`	B^T block evicted on every row
Fixed: `kk` → `jj` → `i` → `j` → `k`	B^T block reused across ALL rows

With the fixed order, the 16 KiB B^T block loads once and serves every row before advancing. This single change reduced cache misses by **16**× and improved IPC by **61%**.

2. Per-child pipes (pipe implementation): Eliminated shared scheduling and single-pipe serialization.

3. Compiler flags: `-O3 -march=native -funroll-loops` enables AVX2 auto-vectorization of the inner accumulation loop. `restrict` pointers guarantee no aliasing.

4 Scalability Results

Processes pinned to cores via `taskset -c 0-$(N-1)`, $N = 5000$.

Table 2: Scalability (N=5000, optimized)

Cores	Shmem (s)	Pipe (s)	Shmem Speedup	Pipe Speedup	Ideal
1	49.00	53.82	1.00×	1.00×	1×
2	25.29	27.51	1.94×	1.96×	2×
4	13.03	14.36	3.76×	3.75×	4×
6	9.18	10.00	5.34×	5.38×	6×
8	7.00	7.71	7.00×	6.98×	8×
16	7.13	8.12	6.87×	6.63×	8×†

†Ideal caps at 8×: cores 8–15 are hyperthreads sharing physical execution units.

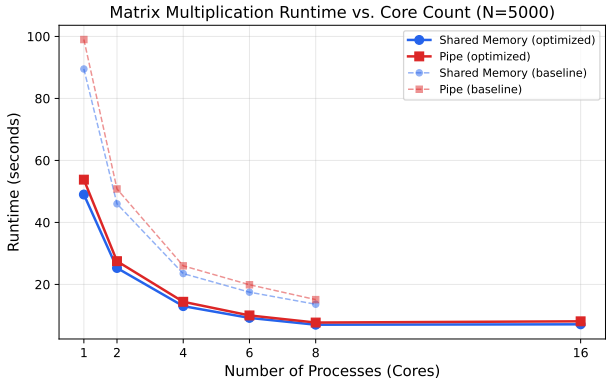


Figure 1: Runtime vs. core count.

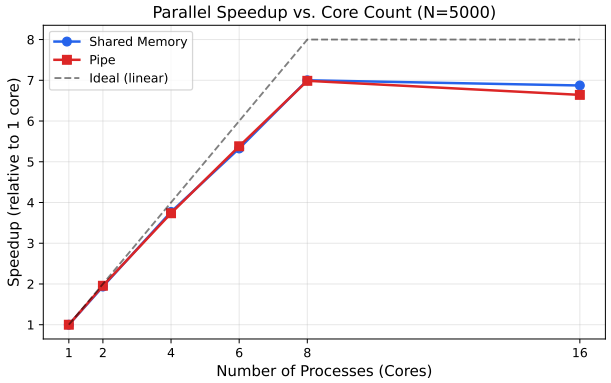


Figure 2: Speedup vs. core count.

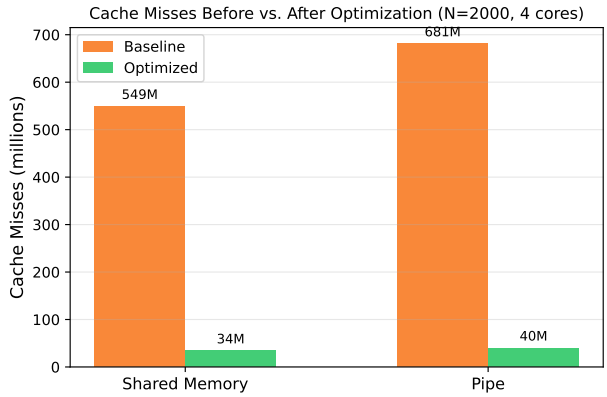


Figure 3: Cache misses: 16× reduction.

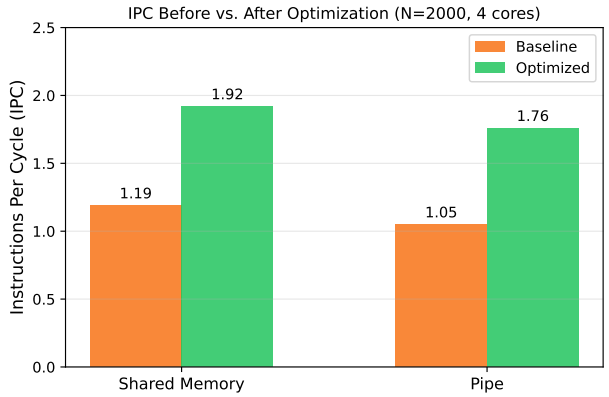


Figure 4: IPC: 1.19 → 1.92 (shmem).

Why 16 processes doesn't help: Since the workload is compute-bound after tiling, hyperthreading cannot extract additional parallelism.

Why shmem beats pipe by ∼10%: Pipes require kernel-mediated copying of all result data (two copies: child → pipe buffer → parent). Shared memory writes directly to the final destination.

Scoreboard Result (`IPC-shmem`, $N = 10000$, 8 cores):

```
Input size: 10000 x 10000
Number of processes: 8
Verification: PASS
Total runtime: 60.505178 seconds
```

5 Program Output

5.1 Shared Memory — Optimized (N=5000, taskset)

```
Input size: 5000 x 5000
Number of processes: 1
Verification: PASS
Total runtime: 49.000189 seconds

Input size: 5000 x 5000
Number of processes: 2
Verification: PASS
Total runtime: 25.292054 seconds

Input size: 5000 x 5000
Number of processes: 4
Verification: PASS
Total runtime: 13.032924 seconds

Input size: 5000 x 5000
Number of processes: 6
Verification: PASS
Total runtime: 9.183621 seconds

Input size: 5000 x 5000
Number of processes: 8
Verification: PASS
Total runtime: 7.003402 seconds

Input size: 5000 x 5000
Number of processes: 16
Verification: PASS
Total runtime: 7.129769 seconds
```

5.2 Pipe — Optimized (N=5000, taskset)

```
Input size: 5000 x 5000
```

```
Number of processes: 1
Verification: PASS
Total runtime: 53.821246 seconds

Input size: 5000 x 5000
Number of processes: 2
Verification: PASS
Total runtime: 27.513679 seconds

Input size: 5000 x 5000
Number of processes: 4
Verification: PASS
Total runtime: 14.361160 seconds

Input size: 5000 x 5000
Number of processes: 6
Verification: PASS
Total runtime: 9.995495 seconds

Input size: 5000 x 5000
Number of processes: 8
Verification: PASS
Total runtime: 7.707416 seconds

Input size: 5000 x 5000
Number of processes: 16
Verification: PASS
Total runtime: 8.116131 seconds
```

5.3 Shared Memory — Baseline (N=5000, no taskset)

```
Input size: 5000 x 5000
Number of processes: 1
Verification: PASS
Total runtime: 89.542042 seconds

Input size: 5000 x 5000
Number of processes: 2
Verification: PASS
Total runtime: 46.011076 seconds

Input size: 5000 x 5000
Number of processes: 4
Verification: PASS
Total runtime: 23.479644 seconds

Input size: 5000 x 5000
Number of processes: 6
Verification: PASS
```

```
Total runtime: 17.498339 seconds

Input size: 5000 x 5000
Number of processes: 8
Verification: PASS
Total runtime: 13.551089 seconds
```

5.4 Pipe — Baseline (N=5000, no taskset)

```
Input size: 5000 x 5000
Number of processes: 1
Verification: PASS
Total runtime: 99.001389 seconds

Input size: 5000 x 5000
Number of processes: 2
Verification: PASS
Total runtime: 50.787100 seconds

Input size: 5000 x 5000
Number of processes: 4
Verification: PASS
Total runtime: 25.966408 seconds

Input size: 5000 x 5000
Number of processes: 6
Verification: PASS
Total runtime: 19.869389 seconds

Input size: 5000 x 5000
Number of processes: 8
Verification: PASS
Total runtime: 15.094266 seconds
```

5.5 Scoreboard Runs (N=10000)

```
Input size: 10000 x 10000
Number of processes: 8
Verification: PASS
Total runtime: 60.505178 seconds

Input size: 10000 x 10000
Number of processes: 16
Verification: PASS
Total runtime: 56.435478 seconds
```

6 Raw perf Output

6.1 Baseline perf stat — Shared Memory (N=2000, 4 processes)

```
 12,068,475,621  cycles                           (57.29%)
 14,335,181,793  instructions     # 1.19 insn/cycle  (71.63%)
    549,561,807  cache-references                   (71.52%)
    549,355,665  cache-misses     # 99.962%         (71.47%)
    560,166,308  L1-dcache-load-misses              (71.49%)
              0  LLC-load-misses                    (71.38%)
      2,355,391  branch-misses                      (57.07%)
             27  context-switches
              0  cpu-migrations
  1.576 seconds time elapsed
```

6.2 Baseline perf stat — Pipe (N=2000, 4 processes)

```
 13,779,899,078  cycles                           (57.15%)
 14,449,337,552  instructions     # 1.05 insn/cycle  (71.49%)
    683,116,623  cache-references                   (71.32%)
    681,927,568  cache-misses     # 99.826%         (71.46%)
    690,723,979  L1-dcache-load-misses              (71.51%)
              0  LLC-load-misses                    (71.52%)
      2,193,090  branch-misses                      (57.20%)
             74  context-switches
              3  cpu-migrations
  1.848 seconds time elapsed
```

6.3 Optimized perf stat — Shared Memory (N=2000, 4 processes)

```
  7,487,007,502  cycles
 14,362,565,858  instructions     # 1.92 insn/cycle
     34,434,499  cache-references
     34,434,499  cache-misses     # 100.000%
     40,019,452  L1-dcache-load-misses
  1.121 seconds time elapsed
```

6.4 Optimized perf stat — Pipe (N=2000, 4 processes)

```
  8,125,788,296  cycles
 14,288,130,418  instructions     # 1.76 insn/cycle
     40,219,588  cache-references
     40,219,588  cache-misses     # 100.000%
     52,637,857  L1-dcache-load-misses
  1.155 seconds time elapsed
```

6.5 Baseline perf report — Shared Memory (N=2000, 4 processes)

```
# Samples: 34K of event 'cycles'
```

```
  97.23%  IPC-shmem  [.] multiply_rows
   0.79%  IPC-shmem  [.] main
   ~1.5%  [unknown]  [k] kernel (page faults, TLB)
```

6.6 Baseline `perf report` — Pipe (N=2000, 4 processes)

```
# Samples: 34K of event 'cycles'
  98.86%  IPC-pipe   [.] main
   ~1.0%  [unknown]  [k] kernel
```

6.7 Optimized `perf report` — Shared Memory (N=2000, 4 processes)

```
# Samples: 16K of event 'cycles'
  95.81%  IPC-shmem  [.] multiply_rows
   1.13%  IPC-shmem  [.] main
   0.15%  libc.so.6  [.] __memset_avx2_unaligned_erms_rtm
   ~1.5%  [unknown]  [k] kernel (page faults)
```

6.8 Optimized `perf report` — Pipe (N=2000, 4 processes)

```
# Samples: 21K of event 'cycles'
  98.17%  IPC-pipe   [.] main (inlined multiply_rows)
   ~1.7%  [unknown]  [k] kernel (pipe buffer, scheduling)
```

6.9 `perf report` Overhead Source Tables

Table 3: Baseline overhead sources (N=2000, 4 processes)

Impl.	Symbol	% Self	Root Cause
Shmem	`multiply_rows`	97.23%	Inner loop stalling on cache misses
Shmem	`main`	0.79%	Init, fork, verify
Shmem	Kernel (unnamed)	∼1.5%	Page faults on shared memory
Pipe	`main`	98.86%	Computation + pipe I/O (all inlined)
Pipe	Kernel (unnamed)	∼1.0%	Pipe buffer mgmt, scheduling

Table 4: Optimized overhead sources (N=2000, 4 processes)

Impl.	Symbol	% Self	Root Cause
Shmem	`multiply_rows`	95.81%	Computation (expected)
Shmem	`main`	1.13%	Init, fork, verify
Shmem	`memset_avx2`	0.15%	Zeroing output matrix C
Shmem	Kernel (unnamed)	∼1.5%	Page faults
Pipe	`main` (inlined)	98.17%	Computation (expected)
Pipe	Kernel (unnamed)	∼1.7%	Pipe buffer mgmt